Node.js: Runtime, Async I/O, and Services

A practical introduction to server-side JavaScript

Purpose: This document is a fictional, self-contained learning resource created for experimenting with PDF text extraction, chunking, embeddings, retrieval, and question answering in a Retrieval-Augmented Generation (RAG) pipeline.

Contents

1. What is Node.js?

2. Modules and package management

3. The event loop and asynchronous work

4. Environment configuration and errors

5. Streams and buffers

6. Security and production readiness

7. Practical project structure

1. What is Node.js?

Node.js is a JavaScript runtime built on the V8 engine. It allows JavaScript to run outside a browser and is commonly used for APIs, command-line tools, real-time services, and background workers.

Node.js uses an event-driven architecture and non-blocking I/O for many operations. This model is effective for workloads that spend time waiting on network, filesystem, or database operations. CPU-heavy work can block the event loop and may need worker threads, child processes, or a separate service.

A simple HTTP server

```
const http = require("node:http");

const server = http.createServer((req, res) =&gt; {
  res.writeHead(200, { "content-type": "text/plain" });
  res.end("Hello from Node.js");
});

server.listen(3000, "127.0.0.1");
```

2. Modules and package management

Node.js supports CommonJS modules with require and module.exports, as well as ECMAScript modules with import and export. A project should use a consistent module strategy and configure package metadata accordingly.

npm is commonly used to install packages and run scripts. package.json records project metadata, dependencies, and scripts; a lockfile records a reproducible dependency tree. Review dependencies, keep them updated, and avoid installing packages from untrusted sources.

Example package.json

```
{
  "name": "sample-node-service",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "node src/server.js",
    "dev": "node --watch src/server.js",
    "test": "node --test"
  }
}
```

3. The event loop and asynchronous work

JavaScript callbacks execute on the event loop. Many I/O operations are initiated asynchronously so the process can continue handling other work while waiting for results. Promises and async/await make asynchronous flows easier to compose, but async/await does not make CPU-intensive code non-blocking by itself.

Avoid synchronous filesystem calls in request handlers when they would stall other requests. Keep callbacks short and move expensive CPU calculations to an appropriate worker or separate process.

Using async/await

```
const fs = require("node:fs/promises");

async function readConfig() {
  try {
    const text = await fs.readFile("./config.json", "utf8");
    return JSON.parse(text);
  } catch (error) {
    if (error.code === "ENOENT") {
      throw new Error("Configuration file is missing");
    }
    throw error;
```

```
  }
}
```

4. Environment configuration and errors

Configuration that differs between environments, such as a port or database URL, should be supplied through environment variables or a secret-management system. Do not commit passwords, tokens, private keys, or production credentials to source control.

Handle expected failures deliberately and send useful, safe error messages to callers. Log enough context to diagnose problems, but avoid logging credentials, session tokens, or sensitive personal data. Attach centralized handling to the application layer where appropriate.

Reading an environment variable

```
const port = Number(process.env.PORT ?? 3000);

if (!Number.isInteger(port) || port &lt; 1 || port &gt; 65535) {
  throw new Error("PORT must be a valid TCP port");
}

console.log(`Service configured for port ${port}`);
```

5. Streams and buffers

A Buffer represents bytes. Streams allow data to be processed incrementally instead of loading an entire file or response into memory. Readable streams provide data, writable streams consume it, and transform streams can modify data as it passes through.

Backpressure is the mechanism that prevents a fast producer from overwhelming a slower consumer. Prefer stream pipelines for large files and network transfers, and handle stream errors.

Streaming a file to a destination

```
const fs = require("node:fs");
const { pipeline } = require("node:stream/promises");

async function copyLargeFile() {
  await pipeline(
    fs.createReadStream("large-input.txt"),
    fs.createWriteStream("large-output.txt")
  );
}
```

6. Security and production readiness

Validate untrusted input at system boundaries, apply least-privilege permissions, use TLS where required, set request-size and timeout limits, and keep dependencies patched. Authentication verifies identity; authorization determines which actions that identity may perform.

Use structured logs and health checks, monitor latency and error rates, and shut down gracefully when the process receives a termination signal. Database operations should use parameterized queries or a safe query builder to reduce injection risk.

7. Practical project structure

A small service might separate route definitions, business logic, data access, configuration, and tests. Keep transport concerns out of domain logic where possible. This separation improves testability and makes it easier to replace a database or HTTP framework.

A typical repository includes src, tests, package.json, a lockfile, and a README. Document required environment variables and provide safe example configuration without real secrets.

Glossary

Runtime — An environment that executes a program.

Event loop — The mechanism that coordinates queued callbacks and asynchronous work.

Non-blocking I/O — I/O that allows the program to continue while an operation is pending.

Promise — An object representing the eventual result or failure of an asynchronous operation.

Buffer — A representation of raw bytes.

Stream — An interface for processing data incrementally.

Environment variable — A configuration value supplied to a running process.

Practice questions for retrieval

Use these questions to test whether your RAG pipeline retrieves the relevant passages rather than answering from general model knowledge.

1. What is Node.js, and what types of workloads is it well suited for?
2. Why can CPU-heavy work be a problem for the event loop?
3. What is the role of package.json and a lockfile?
4. Does async/await make CPU-intensive code non-blocking?
5. Why are streams useful for large files?
6. What is backpressure?
7. List four practices that improve Node.js service security.

End of sample document.